Glowelle - Project documentation

Glowelle is a skincare storefront with an admin back office. The public site shows products, news, offers, and contact information. Customers can register, browse, add items to a cart, and check out. Administrators manage the catalog, orders, offer requests, users, news, and contact details.

This document explains how to install and run the project, what features are included, and how search and pagination work in the admin area and on the storefront.

1. Technical overview

- **Frontend**: React 19, Vite, React Router, React Bootstrap (see src/).
- **Backend**: Node.js with Express; API routes under /api (see server/).
- **Database**: MySQL or MariaDB; connection settings in .env.
- **Authentication**: JSON Web Tokens (JWT); passwords hashed with bcrypt.
- **Development:** the Vite dev server proxies /api to the Express app so the browser calls same-origin /api URLs.

2. Prerequisites

- Node.js 18 or newer and npm.
- A running MySQL or MariaDB server you can connect to from the API.

Trello (team work & tickets)

Glowelle - project board (Trello)
Board: https://trello.com/b/Cime7sDL/glowelle-project

GitHub (source code & version control)

Glowelle - repository (GitHub)
Repository: https://github.com/liza962/Glowelle

3. Installation

3.1 Clone and install dependencies

cd Glowelle

npm install

3.2 Environment file

Copy .env.example to .env in the project root.

Set at least: **MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD, MYSQL_DATABASE, JWT_SECRET, CLIENT_ORIGIN** (e.g. http://localhost:5173), and **PORT** (API port, often 3001).

Optional: **ADMIN_EMAIL** and **ADMIN_PASSWORD** - if no admin user exists, the database init script can create one.

3.3 Database setup

Run the initializer once (or after schema changes). It creates the database and tables if needed and seeds default data where tables are empty:

npm run db:init

Alternatively you can apply server/schema.sql manually in phpMyAdmin or the MySQL client.

4. How to run the application

4.1 Development (recommended)

Start the API and the web UI. You can use two terminals or one combined command.

Terminal 1 - API (default http://localhost:3001):

npm run server

Terminal 2 - Vite dev server (default http://localhost:5173):

npm run dev

Or run both together:

npm run dev:all

Open http://localhost:5173 in your browser.

4.2 Useful commands

- npm run server — API only.
- npm run dev — frontend only.
- npm run build — production build of the SPA to dist/.
- npm run db:init — initialize or update database objects and seed.

4.3 Health checks

GET /api/health - API alive. GET /api/health/db — confirms MySQL connectivity.

5. Using the application

5.1 Public visitors

- **Home**: browse products by category; use search and pagination on the catalog.
- **About, Offers, Contact**: informational pages.
- **News**: read articles from the database.
- **Register / Sign in:** create an account or log in.
- **Signed-in customers (role: user)**: cart, checkout with shipping details, and product purchases stored as orders.
- **Offers page**: book a consultation package; submissions appear in Admin → Offer requests.

5.2 Administrators

Sign in with an admin account. You are taken to the admin panel by default. Use View site in the admin header to return to the storefront.

- **Products**: manage categories and products (CRUD).
- **Orders**: view shop orders (line items, totals). Change order status (pending, confirmed, completed) using the colored status control.
- **Offer requests:** view consultation booking forms from the Offers page; same status workflow as orders.
- **Users:** create and manage user accounts.
- **News:** create and edit articles shown on the public News pages.
- **Contact:** edit the single Contact us block (address lines and map embed URL).

6. Search and pagination

6.1 Storefront - product catalog (Home)

The home page catalog supports filtering by category, searching product text fields, and paging through results so long lists stay manageable. Data comes from the public products API.

6.2 Admin panel

Several admin lists include a search box and pagination (typically 10 items per page):

- Products and Categories (Products admin).
- Users (Users admin).
- News articles (News admin).
- Shop orders (Orders admin).
- Offer requests (Offer requests admin).

How it works: typing in the search field filters rows client-side by matching text in the visible columns (and related fields where applicable). Pagination controls move between pages of the filtered result set. Changing the search text resets to page 1.

7. Security and production notes

- Use a strong **JWT_SECRET** in production.
- Run npm run build and host dist/ behind a static file server or CDN;
- run the Node API separately or behind a reverse proxy.